

Uart_Esp32 - Projektdokumentation - Firmware

Projektname: Uart_Esp32

Dokumentenart: Projektdokumentation

Thema: Firmware für ESP32-WROOM-32 / NodeMCU-32S

Erstellungsdatum: 2026-09-17

Autor / verantwortliche Person: OpenAI ChatGPT (technische Erstellung)

Version: v0.4

Status: Entwurf

Änderungsdatum: 2026-09-17

Ablageort: Projektordner Uart_Esp32/docs

Referenzen: Regeln_Projektdokumentation_PDF_DOCX_Pflicht_(5).pdf; Projektstand v0.3

Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v0.1	2026-09-17	OpenAI ChatGPT	Entwurf	Reduzierter WROOM32-Projektstand erstellt
v0.2	2026-09-17	OpenAI ChatGPT	Entwurf	Projekt in Uart_Esp32 umbenannt und Dokumentationsregeln umgesetzt
v0.3	2026-09-17	OpenAI ChatGPT	Entwurf	Buildfehler durch typensichere Integer-Vergleiche behoben
v0.4	2026-09-17	OpenAI ChatGPT	Entwurf	Arduino-Secrets, NVS-Fallbacks und WLAN-Provisionierung ergänzt

Ziel des Standes v0.4

Der Firmwarestand v0.4 beseitigt die beim Erststart sichtbaren Preferences-Meldungen für noch nicht vorhandene NVS-Keys und ergänzt eine klare Konfigurationshierarchie. Lokale Zugangsdaten können aus einer nicht versionierten `arduino_secrets.h` kommen. Sind keine WLAN-Daten vorhanden, startet der ESP32 im Fallback-AP. Im Webinterface kann anschließend ein gefundenes WLAN ausgewählt werden; SSID und Passwort werden in NVS gespeichert und bei späteren Starts bevorzugt verwendet.

Konfigurationshierarchie

Die Firmware lädt konfigurierbare Werte in folgender Reihenfolge:

1. NVS / Preferences, sofern der konkrete Key bereits existiert.
2. Optionale Datei `include/arduino_secrets.h` bzw. Compile-Defaults aus dieser Datei.
3. Sichere interne Defaults aus `src/ConfigDefaults.h`.

Diese Reihenfolge ist insbesondere für WLAN wichtig: Nach einer Web-Konfiguration bleiben die in NVS gespeicherten Zugangsdaten auch dann erhalten, wenn im Quellcode weiterhin andere Compile-Defaults vorhanden sind.

Vor jedem Lesen von String- und Float-Werten aus NVS wird mit `Preferences::isKey()` geprüft, ob der Key existiert. Dadurch werden bei einem frischen NVS keine irreführenden `NOT_FOUND`-Fehler mehr für `hostname`, `ntp_server`, MQTT-, Batterie-, Deep-Sleep- oder OTA-Werte ausgegeben.

Arduino-Secrets

Die echte Datei `include/arduino_secrets.h` ist optional und wird über `.gitignore` ausgeschlossen. Als Vorlage wird `include/arduino_secrets.example.h` ausgeliefert.

Beispiel:

```
#pragma once
#define UART_WIFI_SSID    "MeinWLAN"
#define UART_WIFI_PASSWORD "MeinPasswort"
#define UART_MQTT_ENABLED true
#define UART_MQTT_HOST    "192.168.1.10"
#define UART_MQTT_USER    "mqtt"
#define UART_MQTT_PASSWORD "geheim"
#define UART_OTA_PASSWORD "geheim"
```

Die gebräuchlichen Arduino-Makros `SECRET_SSID` und `SECRET_PASS` werden zusätzlich als WLAN-Aliase unterstützt. Fehlt die Datei vollständig, kompiliert die Firmware trotzdem. Geheimnisrelevante Fallbackwerte wie WLAN-SSID, WLAN-Passwort, MQTT-Broker und MQTT-Passwort sind intern leer bzw. deaktiviert.

WLAN-Provisionierung

Beim Start gilt:

- Ist eine SSID in NVS gespeichert, wird sie verwendet.
- Andernfalls wird die SSID aus `arduino_secrets.h` verwendet, sofern vorhanden und nicht leer.
- Gibt es keine SSID, startet sofort der Fallback-AP.
- Schlägt die WLAN-Verbindung innerhalb des Startfensters fehl, bleibt ebenfalls der Fallback-AP verfügbar.

Fallback-AP Standard:

- SSID: `Uart_Esp32-Setup`
- IP: `192.168.4.1`
- Passwort: leer / offener AP

Auf `/network` zeigt die Firmware gefundene WLANs mit RSSI und Kennzeichnung offen/geschützt an. Ein WLAN kann ausgewählt oder die SSID manuell eingegeben werden. Beim Speichern werden SSID, Passwort, statische IP/DHCP-Einstellungen, AP-Werte, Hostname und NTP-Server in der NVS-Namespace `netconf` gespeichert. Anschließend startet der ESP32 neu.

Wird eine andere SSID ausgewählt und kein neues Passwort eingegeben, wird das bisherige WLAN-Passwort bewusst verworfen, damit kein Passwort eines alten Netzes auf ein neues Netz angewendet wird.

Projektstruktur

```
Uart_Esp32/
.gitignore
platformio.ini
README.md
include/
  arduino_secrets.example.h
  arduino_secrets.h      # lokal, nicht in Git
src/
  ConfigDefaults.h
```

BatteryMonitor.cpp/.h
 DeepSleepManager.cpp/.h
 MqttManager.cpp/.h
 OtaManager.cpp/.h
 main.cpp
 docs/
 2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.4.*
 2026-09-17 - Uart_Esp32 - Commit-Nachricht - Secrets-und-WLAN-Provisionierung -
 v0.4.txt

Hardware / PlatformIO

Zielplattform ist ESP32-WROOM-32 / NodeMCU-32S:

board = `nodemcu-32s`
framework = `arduino`

Standardumgebung ist `uart_esp32_usb` mit `esptool`. Zusätzlich ist `uart_esp32_ota` mit `espota` vorhanden.

Enthaltene Firmwarefunktionen

- WLAN-Client mit Reconnect
- Fallback Access Point und WLAN-Scan/-Auswahl
- NTP mit CET/CEST und automatischer Sommer-/Winterzeit
- Webinterface
- MQTT mit Statuswerten, Fernsteuerung und Heartbeat
- Deep Sleep mit Intervall, voller Stunde, festen Zeiten, Mixed-Modus, Einmal-Wakeup, Nachtmodus, Batterie- und MQTT-Abhängigkeit
- Browser OTA
- ArduinoOTA / PlatformIO OTA
- HTTP Pull OTA
- NVS / Preferences
- interne ESP32-Temperatur
- Web-Reboot
- ADC1-Batteriemessung als Datenquelle für batterieabhängigen Deep Sleep

Web-Routen

Route	Funktion
/	Hauptstatus
/network	WLAN-Scan/Auswahl, AP, Hostname und NTP
/wifi_rescan	asynchronen WLAN-Scan neu starten
/save_network	WLAN-/Netzwerkdaten in NVS speichern und neu starten
/mqtt	MQTT-Einstellungen und Verbindungstest
/battery	ADC-Batteriemessung
/deepsleep	Deep-Sleep-Konfiguration
/ota	Browser OTA, ArduinoOTA und Pull OTA
/reboot	Neustart per POST

NVS-Verhalten

Die bestehende Namespace `netconf` und die bisherigen Key-Namen bleiben erhalten. Damit bleiben bereits gespeicherte Einstellungen aus v0.3 grundsätzlich kompatibel. Bei einem frischen ESP32 werden fehlende Keys nicht sofort geschrieben; sie werden zunächst aus den Compile-Defaults abgeleitet. Erst eine Änderung über das Webinterface oder eine andere speichernde Funktion erzeugt die entsprechenden NVS-Keys.

Git / Geheimnisse

Die .gitignore enthält insbesondere:

```
.pio/
.vscode/.browse.c_cpp.db*
.vscode/c_cpp_properties.json
.vscode/launch.json
.vscode/ipch/
arduino_secrets.h
.env
secrets.ini
```

arduino_secrets.example.h wird dagegen bewusst versioniert, damit die benötigten Defines dokumentiert bleiben. Eine echte arduino_secrets.h darf keine realen Zugangsdaten im Repository enthalten.

Geänderte Dateien in v0.4

- .gitignore
- README.md
- include/arduino_secrets.example.h
- src/ConfigDefaults.h
- src/main.cpp
- src/BatteryMonitor.cpp
- src/DeepSleepManager.cpp
- src/OtaManager.cpp
- Projektdokumentation v0.4
- Commit-Nachricht v0.4

Prüfung und offene Verifikation

Die Änderungen wurden statisch auf Konsistenz geprüft. Insbesondere wurden alle zuvor gemeldeten String-/Float-NVS-Lesezugriffe so angepasst, dass sie fehlende Keys vor dem eigentlichen Zugriff prüfen. Eine echte PlatformIO-Kompilierung ist in der Erstellungsumgebung weiterhin nicht verfügbar. Der maßgebliche lokale Test lautet daher:

```
pio run -e uart_esp32_usb
pio run -e uart_esp32_usb -t upload
```

Beim ersten Start ohne arduino_secrets.h wird erwartet, dass direkt der AP Uart_Esp32-Setup erscheint und keine Preferences ... NOT_FOUND-Fehler für die optionalen Konfigurationswerte ausgegeben werden.

Auslieferungsform

Entsprechend den Projektregeln werden bereitgestellt:

- vollständiges ZIP mit Root-Ordner Uart_Esp32,
- ZIP mit nur geänderten/neuen Dateien in derselben Projektstruktur,
- Patch-Datei, soweit sinnvoll,
- englische Commit-Nachricht mit Datum, BL_-Titel und vollständiger Liste geänderter Dateien,
- Projektdokumentation als PDF und bearbeitbare DOCX mit identischem Versionsstand v0.4.